PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
ENGENHARIA DE SOFTWARE

EDUARDO REIS
GABRIEL CAVALHEIRO DE OLIVEIRA
GABRIEL RAMOS LUCENA CAMPOS
KAUAN GOMES PAINS
RODOLFO LOURES BUENO BELLE

FECHADURA ELETRÔNICA COM AUTENTICAÇÃO RFID E ADMINISTRAÇÃO REMOTA VIA SERVIDOR HTTP EMBARCADO NO ESP32

CURITIBA
2026

EDUARDO REIS
GABRIEL CAVALHEIRO DE OLIVEIRA
GABRIEL RAMOS LUCENA CAMPOS
KAUAN GOMES PAINS
RODOLFO LOURES BUENO BELLE

FECHADURA ELETRÔNICA COM AUTENTICAÇÃO RFID E ADMINISTRAÇÃO REMOTA VIA SERVIDOR HTTP EMBARCADO NO ESP32

Relatório técnico apresentado à disciplina de Conectividade em Sistemas Ciberfísicos, do curso de Engenharia de Software da Escola Politécnica da Pontifícia Universidade Católica do Paraná, como requisito de avaliação do Trabalho Discente Efetivo (TDE). Turma 2ºA.

Professor: Fábio Bettio.

CURITIBA
2026

RESUMO

Este relatório descreve o projeto, o desenvolvimento e a validação de uma fechadura eletrônica que libera o acesso por meio de cartões RFID e que pode ser administrada pela rede local. O sistema utiliza um microcontrolador ESP32 conectado a um leitor RC522, a um módulo relé e a uma fechadura solenoide de 12 V. O ESP32 compara o identificador único (UID) do cartão com uma lista de credenciais autorizadas, gravada em memória não volátil, e aciona o relé quando o acesso é permitido. Paralelamente, o ESP32 opera como servidor HTTP na porta 80, oferecendo uma interface web protegida por usuário e senha (HTTP Basic Auth), pela qual o usuário consulta o registro de acessos, cadastra e remove cartões e abre a porta remotamente. A escolha do HTTP sobre TCP atende ao requisito de o dispositivo atuar como servidor e garante a entrega confiável dos comandos. Os testes comprovaram a leitura dos cartões, a comunicação cliente-servidor, a persistência da lista após reinicialização e o ciclo completo de negação, cadastro, liberação e revogação de uma credencial, a proteção das rotas por login, a resposta correta de todas as rotas em testes automatizados com cURL e o funcionamento do controle de acesso durante a queda da rede. O trabalho também discute as limitações de segurança da solução, em especial o fato de o sistema autenticar o cartão, e não a pessoa, o que impede o não repúdio dos registros.

Palavras-chave: sistemas ciberfísicos; ESP32; RFID; servidor HTTP; controle de acesso.

SUMÁRIO

1 INTRODUÇÃO....4
2 FUNDAMENTAÇÃO TEÓRICA....5
2.1 Sistemas ciberfísicos....5
2.2 Identificação por radiofrequência (RFID)....5
2.3 Microcontrolador ESP32 e barramento SPI....5
2.4 Modelo cliente-servidor e o protocolo HTTP....6
2.5 Protocolos de transporte: TCP e UDP....6
2.6 HTTP e MQTT....7
2.7 Atuadores: relé e fechadura solenoide....7
2.8 Conceitos de segurança aplicados....7
3 ARQUITETURA DO SISTEMA....8
3.1 Visão geral....8
3.2 Componentes....8
3.3 Ligação elétrica....9
3.4 Comunicação em rede....10
4 DESENVOLVIMENTO....12
4.1 Ferramentas....12
4.2 Montagem do hardware....12
4.3 Firmware....13
4.4 Maquete....14
5 TESTES E VALIDAÇÃO....17
5.1 Metodologia....17
5.2 Testes de hardware....17
5.3 Testes de rede....19
5.4 Teste das rotas com cURL....21
5.5 Teste de falha de conexão....22
5.6 Teste integrado com a fechadura solenoide....23
5.7 Síntese dos testes....25
6 RESULTADOS....26
7 CONCLUSÃO....28
REFERÊNCIAS....29
APÊNDICE A – CÓDIGO-FONTE DO FIRMWARE....31

1 INTRODUÇÃO

O controle de acesso a ambientes ainda é feito, em grande parte, com chaves físicas. Esse modelo tem três limitações conhecidas: uma chave perdida obriga a troca da fechadura, uma cópia pode ser feita sem conhecimento do responsável e não há registro de quem entrou, nem de quando. Sistemas eletrônicos de acesso resolvem esses problemas ao substituir a chave por uma credencial que pode ser cadastrada, revogada e auditada.

A disciplina de Conectividade em Sistemas Ciberfísicos propõe, no Trabalho Discente Efetivo (TDE), o desenvolvimento de um sistema que integre um microcontrolador ESP32, ao menos um sensor, ao menos um atuador e comunicação em rede, com o ESP32 obrigatoriamente atuando como servidor (PUCPR, 2026). Neste trabalho, o sensor é um leitor de cartões RFID, o atuador é um conjunto relé e fechadura solenoide, e a comunicação é feita por um servidor HTTP embarcado no próprio ESP32.

O **objetivo geral** é desenvolver uma fechadura eletrônica que libere o acesso por cartão RFID e que possa ser administrada remotamente por um navegador web. Os objetivos específicos são:

a) ler o identificador único (UID) de cartões RFID com o módulo RC522;
b) comparar o UID lido com uma lista de cartões autorizados e acionar a fechadura por meio de um relé;
c) implementar no ESP32 um servidor HTTP com rotas para abrir a porta, cadastrar e remover cartões e consultar o registro de acessos;
d) manter a lista de cartões em memória não volátil, para que ela sobreviva a reinicializações;
e) validar o funcionamento por testes de hardware, de rede e de ponta a ponta.

Este relatório está organizado da seguinte forma: a seção 2 apresenta a fundamentação teórica; a seção 3 descreve a arquitetura do sistema; a seção 4 detalha o desenvolvimento do hardware, do firmware e da maquete; a seção 5 apresenta os testes e as evidências; a seção 6 discute os resultados; e a seção 7 traz as conclusões, limitações e trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Sistemas ciberfísicos

Sistemas ciberfísicos (*Cyber-Physical Systems*, CPS) integram computação, comunicação e processos físicos. Neles, computadores embarcados monitoram e controlam processos do mundo real, normalmente em laços de realimentação nos quais o processo físico afeta a computação e vice-versa (LEE; SESHIA, 2017). Uma fechadura eletrônica conectada é um exemplo direto: um evento físico (a aproximação de um cartão) é convertido em dado, processado por software e transformado novamente em ação física (a liberação da trava).

2.2 Identificação por radiofrequência (RFID)

A identificação por radiofrequência é uma tecnologia de identificação sem contato, na qual um leitor se comunica com uma etiqueta (*tag*) por meio de campos eletromagnéticos. Nas etiquetas passivas, como as usadas neste projeto, não há bateria: a energia necessária para o funcionamento do chip é obtida do próprio campo gerado pelo leitor (FINKENZELLER, 2010).

O módulo RC522 é baseado no circuito integrado MFRC522, um leitor de 13,56 MHz compatível com o padrão ISO/IEC 14443 A e com cartões da família MIFARE (NXP SEMICONDUCTORS, 2016). Cada cartão possui um identificador único (UID) gravado de fábrica, normalmente com 4 bytes. É esse identificador que o sistema usa como credencial. Cartões e chaveiros de 125 kHz, embora semelhantes na aparência, não são compatíveis com esse leitor.

2.3 Microcontrolador ESP32 e barramento SPI

O ESP32 é um microcontrolador da Espressif Systems com processador de dois núcleos, Wi-Fi e Bluetooth integrados e diversos periféricos de comunicação (ESPRESSIF SYSTEMS, 2024). A presença do Wi-Fi nativo permite que o próprio microcontrolador hospede um servidor de rede, sem a necessidade de um computador intermediário.

A comunicação entre o ESP32 e o leitor RC522 utiliza o barramento SPI (*Serial Peripheral Interface*), um protocolo síncrono com um sinal de relógio (SCK), uma linha de dados do mestre para o periférico (MOSI), uma linha do periférico para o mestre (MISO) e uma linha de seleção (SS ou SDA) (NXP SEMICONDUCTORS, 2016). O ESP32 atua como mestre do barramento.

2.4 Modelo cliente-servidor e o protocolo HTTP

No modelo cliente-servidor, um processo servidor permanece à espera de requisições em uma porta conhecida, e processos clientes iniciam a comunicação ao enviar pedidos (KUROSE; ROSS, 2021). O HTTP (*Hypertext Transfer Protocol*) segue esse modelo: o cliente envia uma requisição com um método e um recurso (rota) e o servidor devolve uma resposta com um código de estado e um conteúdo (FIELDING; NOTTINGHAM; RESCHKE, 2022).

Dois métodos são usados neste projeto. O método **GET** é definido como seguro, ou seja, destinado apenas a obter informações, sem alterar o estado do servidor. O método **POST** é usado para enviar dados que o servidor processa e que podem modificar seu estado (FIELDING; NOTTINGHAM; RESCHKE, 2022). Os códigos de estado informam o resultado: 200 indica sucesso, 303 orienta o cliente a buscar outro recurso, 400 indica requisição inválida, 404 indica recurso não encontrado e 409 indica conflito com o estado atual.

Como no HTTP é sempre o cliente quem inicia a comunicação, o servidor não consegue avisar o navegador quando algo muda. Uma solução simples é o *polling*, em que o cliente repete requisições em intervalos regulares. Uma alternativa mais eficiente é o protocolo WebSocket, que mantém uma conexão bidirecional aberta e permite que o servidor envie dados a qualquer momento (FETTE; MELNIKOV, 2011).

2.5 Protocolos de transporte: TCP e UDP

O HTTP é transportado sobre o TCP (*Transmission Control Protocol*), um protocolo orientado à conexão que garante a entrega dos dados, a ordem dos segmentos e a retransmissão em caso de perda (EDDY, 2022). O UDP (*User Datagram Protocol*), por sua vez, envia datagramas independentes, sem confirmação de entrega nem controle de ordem (POSTEL, 1980). Por isso, o UDP é adequado a fluxos em que perdas ocasionais são toleráveis, como leituras contínuas de sensores, e o TCP é adequado a comandos que não podem se perder nem ser duplicados (TANENBAUM; FEAMSTER; WETHERALL, 2021).

2.6 HTTP e MQTT

O MQTT é um protocolo de mensagens do tipo publicação e assinatura, muito usado em Internet das Coisas, no qual os dispositivos se conectam como clientes a um servidor intermediário chamado *broker* (OASIS, 2019). Nesse modelo, o ESP32 seria um cliente, e não um servidor. Como o regulamento do TDE exige que o ESP32 rode um servidor ativo, que receba conexões de clientes externos (PUCPR, 2026), o HTTP foi escolhido.

2.7 Atuadores: relé e fechadura solenoide

Um relé é um interruptor acionado eletricamente: uma pequena corrente na bobina fecha ou abre contatos capazes de conduzir correntes maiores, isolando o circuito de controle do circuito de potência. A fechadura solenoide é um atuador eletromagnético: ao ser energizada, sua bobina gera um campo que desloca a trava e libera o pino da porta. Como o ESP32 fornece apenas 3,3 V e poucos miliampères em seus pinos, o relé é indispensável para comandar a solenoide de 12 V. Um diodo em paralelo com a bobina da solenoide protege o circuito contra o pico de tensão gerado no desligamento.

2.8 Conceitos de segurança aplicados

Autenticação é a verificação de que uma entidade é quem afirma ser. **Não repúdio** é a garantia de que o autor de uma ação não possa negá-la posteriormente. Em sistemas baseados apenas no UID de um cartão, o que se autentica é o cartão, e não a pessoa que o porta. Como o UID pode ser copiado para outro cartão, o registro de acesso não garante o não repúdio (FINKENZELLER, 2010).

O próprio HTTP define um mecanismo simples de autenticação, o *Basic Authentication*: quando um recurso exige login, o servidor responde com o código 401 e o cabeçalho `WWW-Authenticate`, e o navegador passa a enviar usuário e senha no cabeçalho `Authorization` de cada requisição (RESCHKE, 2015). As credenciais são apenas codificadas em Base64, e não criptografadas. Por isso, o esquema só é seguro quando usado sobre uma conexão cifrada, como o HTTPS (RESCHKE, 2015).

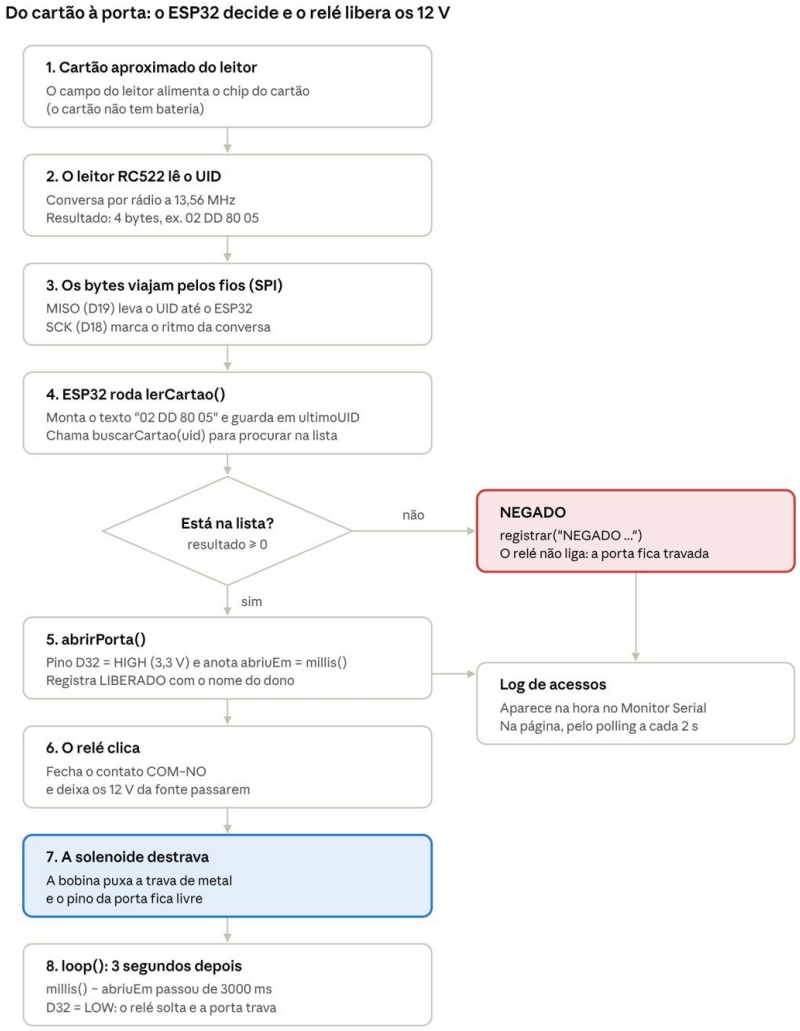
Outro risco comum em aplicações web é o *Cross-Site Scripting* (XSS), em que um dado inserido por um usuário é interpretado pelo navegador como código. A prevenção consiste em codificar os caracteres especiais do HTML antes de exibir dados fornecidos por usuários (OWASP FOUNDATION, 2024).

3 ARQUITETURA DO SISTEMA

3.1 Visão geral

O sistema possui duas entradas e uma única unidade de decisão. A primeira entrada é física: a aproximação de um cartão do leitor. A segunda é pela rede: uma requisição enviada pelo navegador. Em ambos os casos, o ESP32 decide, registra o evento e, quando a decisão é liberar, aciona o relé. A Figura 1 apresenta o caminho completo da informação, desde a aproximação do cartão até a liberação da fechadura.

Figura 1 – Fluxo do cartão até a liberação da fechadura



Fonte: Os autores (2026).

3.2 Componentes

A Tabela 1 relaciona os componentes utilizados e a função de cada um no sistema.

Tabela 1 – Componentes do sistema

Componente	Função no sistema
ESP32 (ESP32-WROOM-32, placa DevKit)	Unidade de processamento: lê o cartão, decide, aciona o relé e executa o servidor HTTP
Leitor RFID RC522 (13,56 MHz)	Sensor: lê o UID dos cartões e envia ao ESP32 pelo barramento SPI
Cartões e chaveiros RFID (MIFARE)	Credenciais de acesso, cada uma com um UID único
Módulo relé de 1 canal (SRD-05VDC-SL-C)	Atuador de comando: liga o circuito de 12 V a partir do sinal de 3,3 V do ESP32
Fechadura solenoide 12 V	Atuador final: destrava a porta quando energizada
Fonte chaveada 12 V / 2 A	Alimentação exclusiva da fechadura
Diodo 1N4007	Proteção contra o pico de tensão da bobina da solenoide
Protoboard e jumpers	Interligação dos módulos sem solda

Fonte: Os autores (2026).

3.3 Ligação elétrica

O sistema é dividido em dois circuitos. O circuito de **controle**, alimentado pela porta USB, reúne o ESP32, o leitor e a bobina do relé. O circuito de **potência**, de 12 V, alimenta apenas a fechadura. Os dois circuitos se encontram somente no relé, o que impede que a tensão de 12 V alcance o ESP32. A Tabela 2 apresenta a pinagem utilizada.

Tabela 2 – Pinagem do sistema

Origem	Destino	Função
RC522 SDA (SS)	ESP32 GPIO 5	Seleção do periférico SPI
RC522 SCK	ESP32 GPIO 18	Relógio do barramento SPI
RC522 MOSI	ESP32 GPIO 23	Dados do ESP32 para o leitor
RC522 MISO	ESP32 GPIO 19	Dados do leitor para o ESP32
RC522 RST	ESP32 GPIO 27	Reinicialização do leitor
RC522 3.3V / GND	ESP32 3V3 / GND	Alimentação do leitor (3,3 V)
Relé S	ESP32 GPIO 32	Sinal de acionamento do relé
Relé + / −	ESP32 VIN (5 V) / GND	Alimentação da bobina do relé
Fonte 12 V (+)	Relé COM	Entrada do circuito de potência
Relé NO	Solenoide (fio vermelho)	Energiza a fechadura somente com o relé acionado
Solenoide (fio preto)	Fonte 12 V (−)	Retorno do circuito de potência

Fonte: Os autores (2026).

3.4 Comunicação em rede

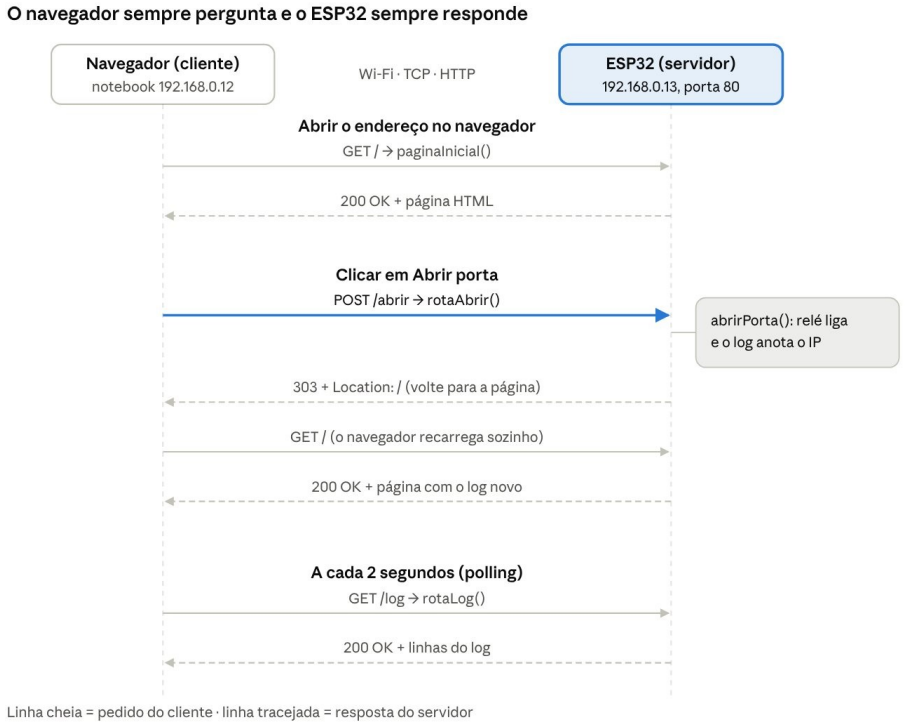
O ESP32 conecta-se à rede Wi-Fi no modo estação, recebe um endereço IP do roteador e inicia um servidor HTTP na porta 80. O navegador de qualquer dispositivo da mesma rede atua como cliente. A Tabela 3 descreve as rotas implementadas, e a Figura 2 mostra a troca de mensagens entre o cliente e o servidor.

Tabela 3 – Rotas HTTP do servidor

Rota	Método	Ação	Respostas
/	GET	Envia a página com estado, cartões, cadastro e log	200, 401
/abrir	POST	Destrava a fechadura por 3 segundos e registra o IP do cliente	303, 401
/cadastrar	POST	Adiciona um cartão (UID e nome) à lista e grava na flash	303, 400, 409, 507
/apagar	POST	Remove um cartão da lista e grava na flash	303, 404
/log	GET	Devolve as linhas do log (usado pelo polling)	200
/ultimo	GET	Devolve o UID do último cartão lido	200

Fonte: Os autores (2026).

Figura 2 – Troca de mensagens entre o navegador e o ESP32



Fonte: Os autores (2026).

Todas as rotas exigem autenticação. Sem o usuário e a senha corretos, o servidor responde com o código 401 e não executa a ação.

As ações que alteram o estado do sistema usam o método POST, e as consultas usam GET. Dessa forma, a porta não pode ser aberta apenas pelo acesso a um endereço na barra do navegador.

4 DESENVOLVIMENTO

4.1 Ferramentas

O firmware foi desenvolvido em C/C++ na Arduino IDE 1.8.19, com o pacote de placas ESP32 da Espressif (ESPRESSIF SYSTEMS, 2025) e a biblioteca MFRC522 (BALBOA, 2024). Foram usadas também as bibliotecas WiFi, WebServer e Preferences, que fazem parte do pacote do ESP32. A maquete foi modelada em Python e impressa em PLA em uma impressora 3D Bambu Lab A1.

4.2 Montagem do hardware

A montagem foi feita em etapas, com teste de cada módulo antes da integração, conforme recomendado no regulamento do TDE (PUCPR, 2026). A Figura 3 mostra os componentes adquiridos e a Figura 4 mostra o ESP32 ligado ao leitor RC522 na protoboard.

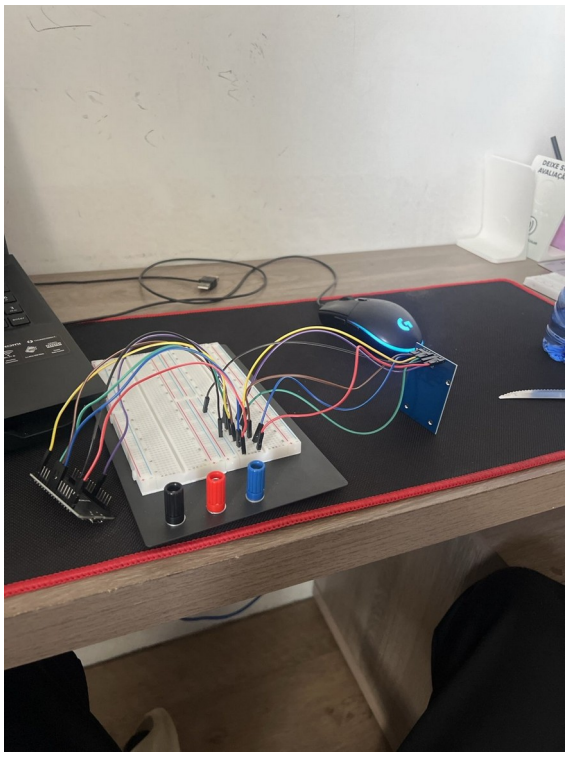
Uma dificuldade prática foi a largura da placa do ESP32, que não deixava furos livres dos dois lados de uma protoboard comum. A solução foi ligar os jumpers do tipo macho-fêmea diretamente nos pinos do ESP32 e usar a protoboard apenas como ponto de conexão.

Figura 3 – Componentes do projeto



Fonte: Os autores (2026).

Figura 4 – ESP32 e leitor RC522 montados na protoboard



Fonte: Os autores (2026).

4.3 Firmware

O firmware completo está no Apêndice A. Ele é organizado em blocos: configuração, funções de apoio, persistência, controle da fechadura, rotas HTTP e as funções `setup()` e `loop()`.

Leitura e decisão. A função `lerCartao()` verifica se há um cartão no campo do leitor, converte os bytes do UID em texto hexadecimal (por exemplo, `02 DD 80 05`) e procura esse texto na lista de autorizados. Se o cartão for encontrado, a função `abrirPorta()` coloca o pino 32 em nível alto, o que aciona o relé; caso contrário, o evento é registrado como negado.

```
int i = buscarCartao(uid);
if (i >= 0) abrirPorta("Cartao " + uid + " (" + nomes[i] + ")");
else        registrar("NEGADO - cartao desconhecido " + uid);
```

Temporização não bloqueante. A porta fica destravada por 3 segundos. Em vez da função `delay()`, que suspenderia todo o programa durante a espera, o firmware registra o instante de abertura com `millis()` e, a cada volta do `loop()`, verifica se o tempo já passou. Assim, o servidor continua respondendo e o leitor continua ativo enquanto a porta está aberta.

```
void loop() {
  server.handleClient();
  lerCartao();
  if (portaAberta && millis() - abriuEm >= TEMPO_ABERTA) {
```

```
    digitalWrite(RELAY_PIN, LOW);
    portaAberta = false;
  }
}
```

Persistência. A lista de cartões é gravada na memória flash com a biblioteca Preferences, a cada cadastro ou remoção, e é carregada no início do programa. O registro de acessos (log) permanece apenas na memória RAM, com capacidade para os 30 eventos mais recentes, para evitar o desgaste da flash por escritas frequentes. Os horários são obtidos pela internet por meio do protocolo NTP (MILLS et al., 2010).

Interface web. A página é gerada pelo próprio ESP32 a cada requisição. Ela exibe o estado da fechadura, a lista de cartões com botão de remoção, o formulário de cadastro e o log. O botão "Usar último lido" consulta a rota `/ultimo` e preenche o UID automaticamente. Um script na página consulta a rota `/log` a cada 2 segundos (*polling*), o que faz o log atualizar sem recarregar a página. Os nomes digitados passam pela função `escapar()`, que impede ataques de XSS.

Autenticação. Cada rota começa chamando a função `autorizado()`, que verifica o usuário e a senha enviados pelo navegador no padrão HTTP Basic Auth. Se estiverem errados ou ausentes, o servidor responde 401 e o navegador exibe a janela de login. Tentativas com senha errada são registradas no log com o IP de origem. Como a página usa *polling*, o navegador reaproveita as credenciais já informadas nas consultas seguintes.

```
bool autorizado() {
  if (server.authenticate(WEB_USUARIO, WEB_SENHA)) return true;
  if (server.hasHeader("Authorization"))
    registrar("LOGIN RECUSADO - " + server.client().remoteIP().toString());
  server.requestAuthentication(BASIC_AUTH, "Fechadura", "Acesso negado: login
necessario");
  return false;
}
```

Validações. A rota de cadastro rejeita campos vazios (código 400), cartões já cadastrados (409) e cadastros acima do limite de 20 cartões (507). A rota de remoção responde 404 para cartões inexistentes.

4.4 Maquete

Para demonstrar o funcionamento, foi modelada e impressa em 3D uma maquete composta por base, parede com batente, porta com dobradiça e suporte da fechadura. As medidas da fechadura, do pino e do leitor foram estimadas a partir de fotografias com um cartão de dimensões padronizadas (85,6 mm × 54 mm) como referência. O suporte foi impresso primeiro, para validar as medidas (Figura 7). O leitor

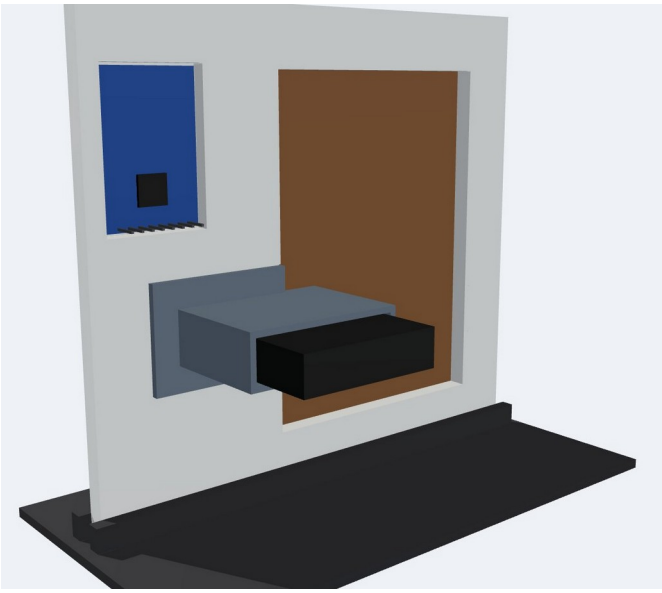
RC522 fica atrás da parede e lê os cartões através de 2 mm de plástico. As Figuras 5 e 6 mostram o modelo da maquete montada.

Figura 5 – Modelo da maquete: vista frontal



Fonte: Os autores (2026).

Figura 6 – Modelo da maquete: vista traseira, com o leitor e a fechadura



Fonte: Os autores (2026).

Figura 7 – Suporte impresso com a fechadura encaixada



Fonte: Os autores (2026).

5 TESTES E VALIDAÇÃO

5.1 Metodologia

Os testes seguiram a ordem de dependência do sistema: primeiro cada componente isoladamente, depois a comunicação em rede e, por fim, o funcionamento de ponta a ponta. As evidências foram registradas por capturas do Monitor Serial da Arduino IDE, do navegador e por fotografias.

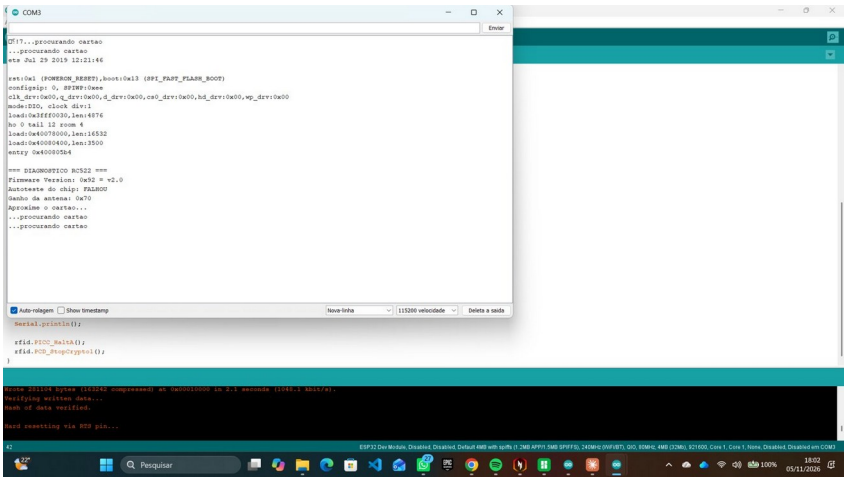
5.2 Testes de hardware

ESP32. O envio do firmware foi confirmado pela mensagem de verificação de integridade da gravação. Inicialmente a porta serial não aparecia na Arduino IDE porque o dispositivo estava desabilitado no Gerenciador de Dispositivos do Windows; após habilitá-lo, a placa foi reconhecida na porta COM3.

Relé. Um programa de teste alternou o pino 32 entre os níveis alto e baixo a cada 2 segundos. O relé respondeu com o acionamento audível e o indicador luminoso a cada mudança.

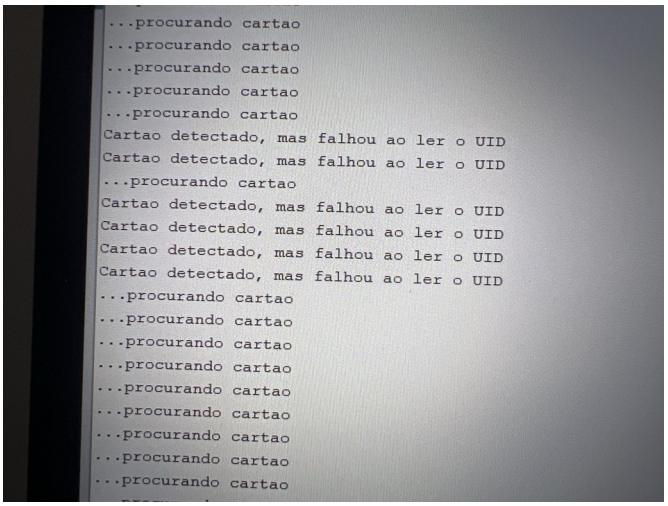
Leitor RC522. O leitor respondeu à consulta de versão (0x92), o que comprovou a comunicação SPI, mas não detectava o cartão e o chaveiro do kit. Para isolar a causa, foi feito um diagnóstico em três passos: (a) execução do autoteste interno do MFRC522, que falhou; (b) repetição com o leitor encaixado diretamente na protoboard, eliminando os fios longos, com o mesmo resultado (Figura 8); e (c) teste com outro cartão de aproximação, que foi detectado, mas sem leitura completa do UID (Figura 9). Ao testar novos cartões e chaveiros MIFARE, o leitor passou a ler os UIDs de forma imediata e repetível (Figura 10). Concluiu-se que a falha estava nas etiquetas originais, e não no leitor nem na ligação. A falha no autoteste, comum em módulos de baixo custo, não impediu o funcionamento.

Figura 8 – Diagnóstico do leitor: comunicação ativa e autoteste com falha



Fonte: Os autores (2026).

Figura 9 – Cartão detectado, mas sem leitura completa do UID



Fonte: Os autores (2026).

Figura 10 – Leitura dos UIDs do novo cartão (02 DD 80 05) e do chaveiro (11 F8 46 A2)

```
ets Jul 29 2019 12:21:46

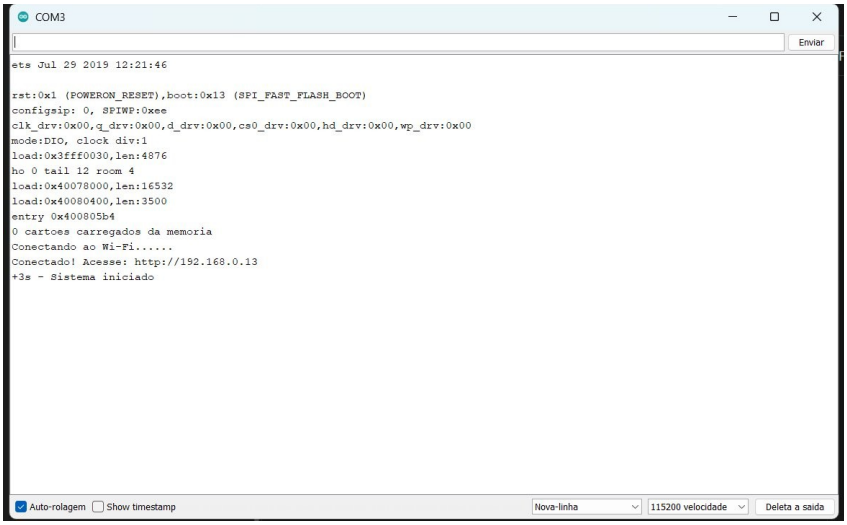
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4876
ho 0 tail 12 room 4
load:0x40078000,len:16532
load:0x40080400,len:3500
entry 0x400805b4
RC522 OK (versao 0x92). Aproxime o cartao...
UID: 02 DD 80 05
UID: 02 DD 80 05
UID: 02 DD 80 05
UID: 11 F8 46 A2
UID: 11 F8 46 A2
UID: 11 F8 46 A2
UID: 02 DD 80 05
UID: 11 F8 46 A2
```

Fonte: Os autores (2026).

5.3 Testes de rede

Conexão Wi-Fi. O ESP32 conectou-se à rede de 2,4 GHz e recebeu o endereço 192.168.0.13 (Figura 11).

Figura 11 – ESP32 conectado à rede e servidor iniciado



Fonte: Os autores (2026).

Abertura remota. O acesso à página pelo navegador do notebook (192.168.0.12) e o clique em "Abrir porta" acionaram o relé. O evento foi registrado com data, hora e o IP do cliente (Figura 12).

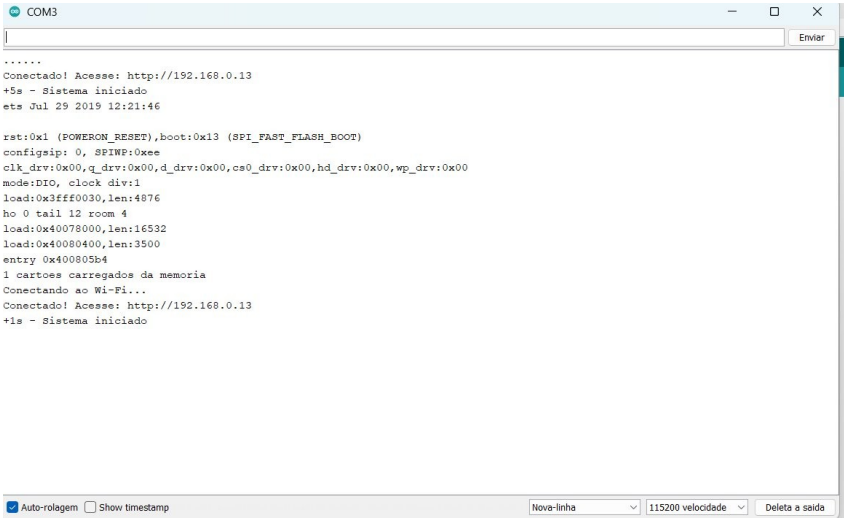
Figura 12 – Abertura remota registrada no log



Fonte: Os autores (2026).

Persistência. Após o cadastro de um cartão e a reinicialização do ESP32, o Monitor Serial informou "1 cartoes carregados da memoria", e o cartão continuou na lista (Figura 13).

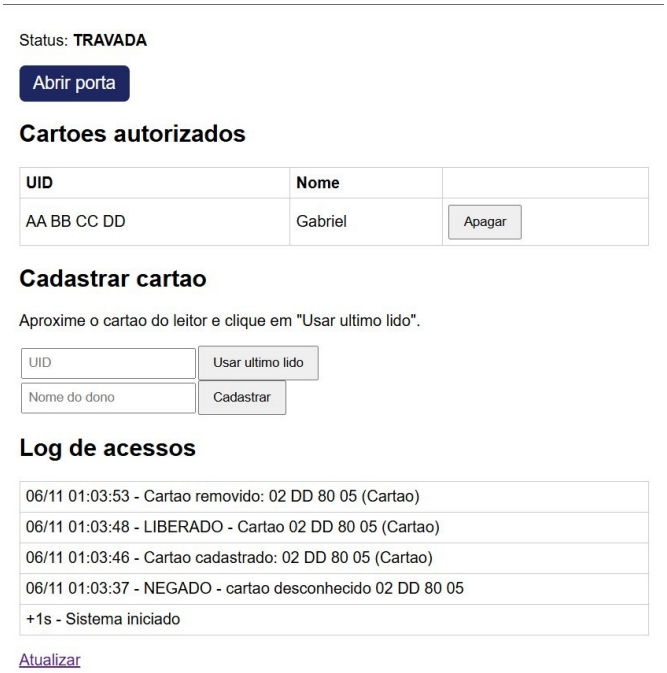
Figura 13 – Lista de cartões mantida após a reinicialização



Fonte: Os autores (2026).

Ciclo completo. Foi executado o ciclo de vida completo de uma credencial: um cartão desconhecido foi negado, cadastrado pelo navegador com o botão "Usar último lido", liberado na leitura seguinte e removido. Os quatro eventos ocorreram em 16 segundos e apareceram no log sem recarregar a página, pelo mecanismo de *polling* (Figura 14).

Figura 14 – Ciclo completo: negado, cadastrado, liberado e removido



Fonte: Os autores (2026).

5.4 Teste das rotas com cURL

Para validar a API independentemente do navegador, foi preparado um roteiro de testes com a ferramenta cURL, que envia requisições HTTP diretamente ao servidor (Tabela 4).

Tabela 4 – Roteiro de testes das rotas com cURL

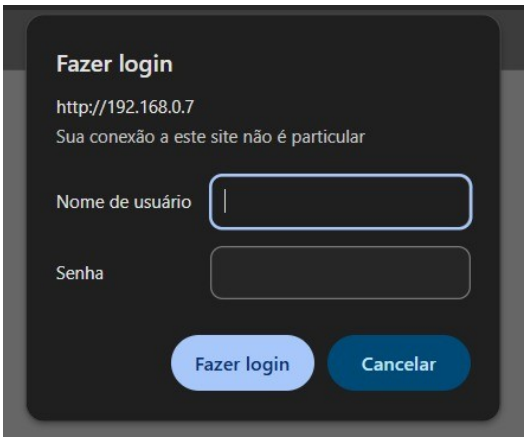
Nº	Requisição	Resultado esperado	Resultado obtido
0a	GET / sem login	401, página não exibida	401 Unauthorized
0b	GET / com senha errada	401 e LOGIN RECUSADO no log	401 e evento registrado no log
0c	POST /abrir sem login	401, a porta não abre	401, relé não acionado
1	GET / (com login)	200 e página HTML	200 (1.832 bytes em 43 ms)
2	GET /ultimo	200 e UID do último cartão	200 (02 DD 80 05)
3	POST /cadastrar (cartão de teste)	303 e cartão na lista	303 See Other
4	POST /cadastrar (mesmo cartão)	409 Cartão já cadastrado	409 Conflict
5	POST /cadastrar (campos vazios)	400	400 Bad Request
6	POST /abrir	303 e acionamento do relé	303 e relé acionado
7	GET /abrir (método errado)	404, a porta não abre	404 Not Found
8	POST /apagar (cartão de teste)	303 e cartão removido	303 See Other
9	POST /apagar (cartão inexistente)	404	404 Not Found
10	GET /log	200 e linhas do log	200 com todos os eventos dos testes
11	20 requisições seguidas a /log	Todas com 200; registrar tempo médio	20 de 20 com 200; média de 44 ms (20 a 71 ms)

Fonte: Os autores (2026).

Todas as 15 requisições retornaram o código esperado. As requisições sem credenciais ou com senha errada foram recusadas com 401 sem executar nenhuma ação, e a tentativa com senha errada foi registrada no log com o IP de origem. O GET em /abrir retornou 404, o que confirma que apenas o método POST aciona a fechadura. No teste de carga, as 20 requisições seguidas foram atendidas sem falhas, com tempo médio de resposta de 44 ms. O resultado completo foi salvo em um arquivo de texto pelo próprio script de testes (testar_rotas.bat), disponível no repositório do projeto.

A Figura 15 mostra a janela de login exibida pelo navegador ao acessar a página, e a Figura 16 mostra o log com as tentativas de acesso recusadas.

Figura 15 – Janela de login solicitada pelo navegador (HTTP Basic Auth)



Fonte: Os autores (2026).

Figura 16 – Log com as tentativas de login recusadas



Fonte: Os autores (2026).

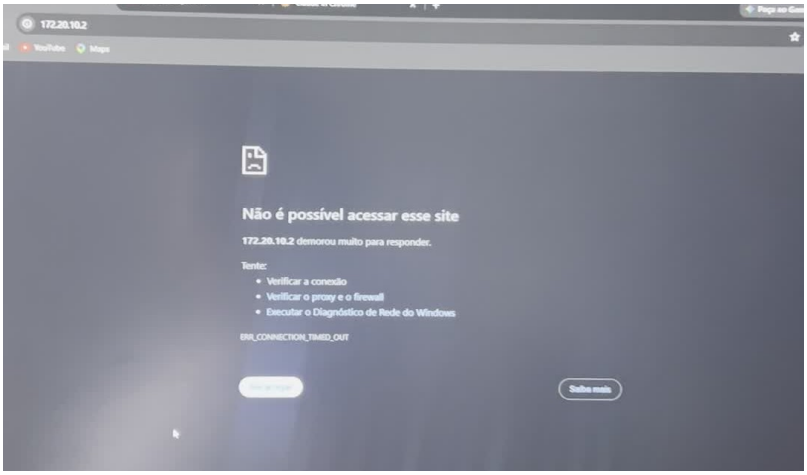
5.5 Teste de falha de conexão

O sistema foi projetado para que a decisão de acesso dependa apenas do ESP32. Para verificar esse comportamento, o roteador deve ser desligado com o sistema em funcionamento e um cartão cadastrado deve ser aproximado do leitor. O resultado esperado é a liberação da porta, mesmo sem rede, e a indisponibilidade apenas das funções remotas.

O teste foi feito com o ESP32 conectado ao roteador de um celular (endereço 172.20.10.2). Com o sistema em funcionamento, o roteador foi desligado. Nessa condição, o cartão cadastrado foi aproximado do leitor e o relé foi acionado normalmente, enquanto o chaveiro não cadastrado foi negado. A página, como

esperado, ficou inacessível (Figura 17). Após religar o roteador, o ESP32 reconectou-se automaticamente, e o log exibiu todos os eventos ocorridos durante a falta de rede, com o horário correto, já que o relógio havia sido sincronizado por NTP antes da queda (Figura 18).

Figura 17 – Página inacessível no navegador durante a queda da rede



Fonte: Os autores (2026).

Figura 18 – Eventos de acesso registrados durante o teste sem Wi-Fi



Fonte: Os autores (2026).

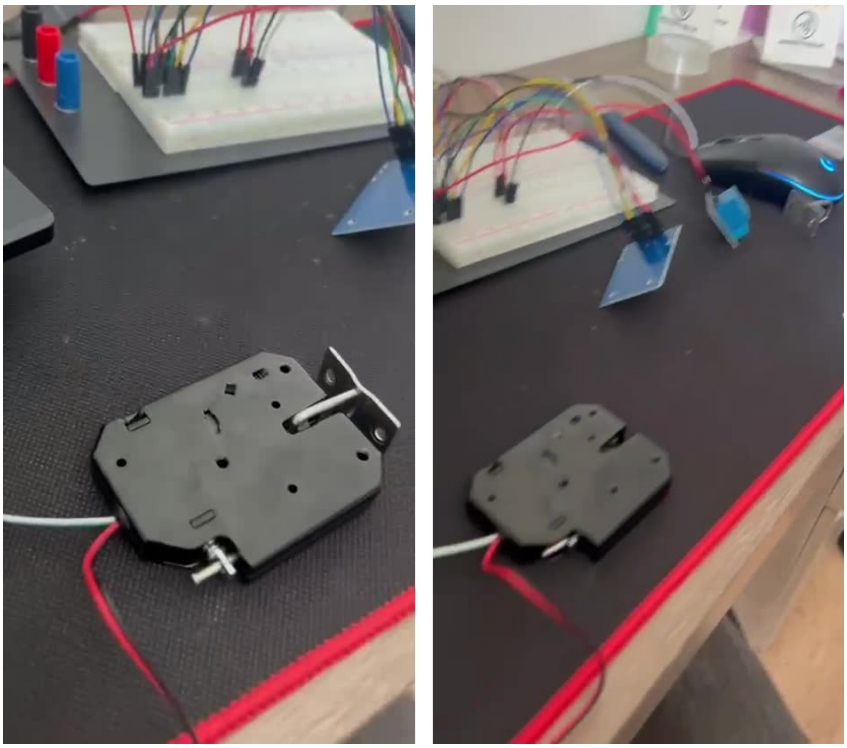
Uma limitação identificada é que o firmware aguarda a conexão Wi-Fi durante a inicialização. Se o ESP32 for ligado sem a rede disponível, ele permanece nessa espera e o leitor não é atendido. Portanto, a independência da rede vale para quedas que ocorrem com o sistema já em funcionamento.

5.6 Teste integrado com a fechadura solenoide

Antes da integração com o relé, a fechadura foi testada diretamente na fonte de 12 V. Com o pino da porta encaixado, a fechadura o manteve preso sem energia e o liberou ao ser energizada, o que confirmou o comportamento *fail-secure* (trancada na

falta de energia) (Figura 19). O primeiro teste falhou por mau contato do plugue da fonte no adaptador de bornes, resolvido com o encaixe completo do plugue.

Figura 19 – Fechadura segurando o pino sem energia (esquerda) e após liberá-lo ao ser energizada (direita)



Fonte: Os autores (2026).

Durante a gravação desse teste, a fechadura permaneceu energizada por cerca de um minuto, por descuido. Esse modelo é projetado para pulsos curtos, e o aquecimento danificou o mecanismo: o gancho deixou de reter o pino e, em seguida, a bobina deixou de responder. Uma nova unidade do mesmo modelo foi adquirida para substituí-la. No uso normal esse risco não ocorre, porque o firmware energiza a fechadura apenas durante o tempo de abertura e a desliga automaticamente.

O circuito de acionamento, com o ESP32, o relé e a fonte, foi validado em todos os testes anteriores pelo acionamento do relé. A nova fechadura foi então ligada aos contatos COM e NO do relé, com o diodo 1N4007 em paralelo com a bobina, e utilizada na apresentação do projeto.

Com o relé no circuito, a fonte de 12 V pode permanecer conectada à tomada: o contato NO só fecha quando o firmware aciona o relé, e a fechadura recebe energia apenas durante o tempo de abertura. A precaução de desconectar a fonte logo após cada teste vale somente para a fechadura ligada diretamente na fonte, sem o relé, que foi a situação em que a primeira unidade se danificou.

5.7 Síntese dos testes

Tabela 5 – Síntese dos testes realizados

Teste	**Situação**
Gravação do firmware no ESP32	Aprovado
Acionamento do relé pelo GPIO 32	Aprovado
Comunicação SPI com o RC522	Aprovado
Leitura de UID (cartão e chaveiro MIFARE)	Aprovado
Conexão Wi-Fi e servidor HTTP	Aprovado
Abertura remota pelo navegador	Aprovado
Cadastro e remoção de cartões	Aprovado
Persistência da lista após reinicialização	Aprovado
Ciclo completo com atualização automática do log	Aprovado
Login da interface web (401 sem credenciais)	Aprovado
Rotas com cURL (15 requisições e teste de carga)	Aprovado
Funcionamento sem Wi-Fi	Aprovado (com o sistema já iniciado)
Fechadura solenoide direto na fonte	Aprovado; unidade danificada por superaquecimento e substituída

Fonte: Os autores (2026).

6 RESULTADOS

O sistema cumpriu os requisitos obrigatórios do TDE, conforme a Tabela 6.

Tabela 6 – Atendimento aos requisitos do TDE

Requisito (PUCPR, 2026)	**Como foi atendido**
ESP32 como microcontrolador	ESP32-WROOM-32 em placa DevKit
Ao menos 1 sensor	Leitor RFID RC522
Ao menos 1 atuador	Relé e fechadura solenoide de 12 V
ESP32 operando como servidor	Servidor HTTP na porta 80
Consultar dados pela rede	Rotas /log e /ultimo e a página principal
Enviar comandos pela rede	Rotas /abrir, /cadastrar e /apagar
Visualizar o estado do sistema	Estado TRAVADA/DESTRAVADA e log atualizado por polling

Fonte: Os autores (2026).

A leitura dos cartões ocorre com a etiqueta a poucos centímetros do leitor, inclusive através da parede de plástico da maquete. A maioria das requisições pelo navegador foi respondida de forma imediata. Em algumas ocasiões, a primeira requisição após um intervalo levou de 2 a 5 segundos, comportamento associado ao atendimento de uma conexão por vez pelo servidor e às conexões antecipadas abertas pelo navegador. A desativação do modo de economia de energia do Wi-Fi reduziu o tempo de conexão inicial de cerca de 5 para 1 segundo.

Os principais problemas encontrados e as soluções adotadas estão na Tabela 7.

Tabela 7 – Problemas encontrados e soluções

Problema	**Causa**	**Solução**
Porta COM indisponível na Arduino IDE	Dispositivo desabilitado no Windows	Habilitação no Gerenciador de Dispositivos
ESP32 sem furos livres na protoboard	Largura da placa	Jumpers macho-fêmea direto nos pinos
Cartões do kit não eram lidos	Etiquetas incompatíveis ou defeituosas	Novos cartões MIFARE de 13,56 MHz
Conexão Wi-Fi lenta	Modo de economia de energia	WiFi.setSleep(false)
Log não atualizava ao ler cartão	No HTTP, o servidor não inicia a comunicação	Polling da rota /log a cada 2 s
Fechadura não acionava na fonte	Plugue mal encaixado no adaptador	Encaixe completo do plugue
Fechadura danificada	Energizada por cerca de 1 minuto em teste	Substituição; testes diretos limitados a 1 ou 2 s

Problema	Causa	Solução
IP do ESP32 mudava a cada conexão	Endereço atribuído por DHCP	Consulta do IP no Monitor Serial

Fonte: Os autores (2026).

7 CONCLUSÃO

Este trabalho desenvolveu uma fechadura eletrônica que combina autenticação por cartão RFID e administração remota por um servidor HTTP embarcado no ESP32. O sistema demonstrou, na prática, os conceitos centrais da disciplina: a integração de sensor e atuador a um microcontrolador, o modelo cliente-servidor, o uso do HTTP sobre TCP e a escolha consciente de protocolos. Em relação a uma chave física, a solução oferece revogação imediata de credenciais, registro de acessos e uma credencial individual por usuário.

O sistema possui limitações que precisam ser reconhecidas. A principal é que ele **autentica o cartão, e não a pessoa**: como o UID pode ser clonado, o registro de acesso aponta o dono do cartão original, e não necessariamente quem o utilizou, o que impede o não repúdio. Além disso, embora a interface web exija usuário e senha, o login usa HTTP Basic Auth sem HTTPS: as credenciais trafegam sem criptografia e podem ser capturadas por quem monitora a rede, e a senha é fixa no código. O log é mantido apenas em memória volátil e se perde quando o ESP32 é desligado. O servidor atende uma conexão por vez, o que pode gerar atrasos ocasionais. Por fim, o ESP32 precisa da rede disponível ao ser ligado, pois aguarda a conexão Wi-Fi antes de iniciar o leitor.

Como trabalhos futuros, propõe-se: a adoção de cartões MIFARE DESFire, que provam o conhecimento de uma chave secreta sem revelá-la; a inclusão de um PIN como segundo fator de autenticação; o uso de HTTPS e de senhas configuráveis e armazenadas com *hash*; a gravação do log em cartão SD ou em servidor externo; a substituição do *polling* por WebSocket; a inicialização do leitor independente da conexão Wi-Fi; um pulso de acionamento mais curto e um intervalo mínimo entre aberturas, para limitar o aquecimento da fechadura; e o uso dos fios de sensor da fechadura para informar se a porta está fisicamente aberta ou fechada.

O código-fonte, os diagramas de arquitetura e de ligação, os scripts de teste e as evidências do projeto estão disponíveis no repositório do grupo no GitHub, cujo endereço acompanha esta entrega.

REFERÊNCIAS

BALBOA, M. **MFRC522**: Arduino RFID library for MFRC522. Versão 1.4.12. [*S. l.*]: GitHub, 2024. Disponível em: https://github.com/miguelbalboa/rfid. Acesso em: 6 nov. 2026.

EDDY, W. (ed.). **Transmission Control Protocol (TCP)**. RFC 9293. [*S. l.*]: Internet Engineering Task Force, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9293. Acesso em: 6 nov. 2026.

ESPRESSIF SYSTEMS. **Arduino-ESP32 documentation**. Xangai: Espressif Systems, 2025. Disponível em: https://docs.espressif.com/projects/arduino-esp32/en/latest/. Acesso em: 6 nov. 2026.

ESPRESSIF SYSTEMS. **ESP32 series datasheet**. Xangai: Espressif Systems, 2024. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32*datasheet*en.pdf. Acesso em: 6 nov. 2026.

FETTE, I.; MELNIKOV, A. **The WebSocket Protocol**. RFC 6455. [*S. l.*]: Internet Engineering Task Force, 2011. Disponível em: https://www.rfc-editor.org/rfc/rfc6455. Acesso em: 6 nov. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). **HTTP Semantics**. RFC 9110. [*S. l.*]: Internet Engineering Task Force, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110. Acesso em: 6 nov. 2026.

FINKENZELLER, K. **RFID handbook**: fundamentals and applications in contactless smart cards, radio frequency identification and near-field communication. 3. ed. Chichester: Wiley, 2010.

KUROSE, J. F.; ROSS, K. W. **Redes de computadores e a Internet**: uma abordagem top-down. 8. ed. São Paulo: Pearson, 2021.

LEE, E. A.; SESHIA, S. A. **Introduction to embedded systems**: a cyber-physical systems approach. 2. ed. Cambridge: MIT Press, 2017.

MILLS, D. et al. **Network Time Protocol Version 4**: protocol and algorithms specification. RFC 5905. [*S. l.*]: Internet Engineering Task Force, 2010. Disponível em: https://www.rfc-editor.org/rfc/rfc5905. Acesso em: 6 nov. 2026.

NXP SEMICONDUCTORS. **MFRC522**: standard performance MIFARE and NTAG frontend. Rev. 3.9. Eindhoven: NXP Semiconductors, 2016. Disponível em: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf. Acesso em: 6 nov. 2026.

OASIS. **MQTT Version 5.0**. OASIS Standard. [*S. l.*]: OASIS, 2019. Disponível em: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html. Acesso em: 6 nov. 2026.

OWASP FOUNDATION. **Cross Site Scripting Prevention Cheat Sheet**. [*S. l.*]: OWASP, 2024. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Cross*Site*Scripting*Prevention*Cheat_Sheet.html. Acesso em: 6 nov. 2026.

POSTEL, J. **User Datagram Protocol**. RFC 768. [*S. l.*]: Internet Engineering Task Force, 1980. Disponível em: https://www.rfc-editor.org/rfc/rfc768. Acesso em: 6 nov. 2026.

PUCPR. **Manual e regulamento do TDE**: disciplina Conectividade em Sistemas Ciberfísicos. Curitiba: Pontifícia Universidade Católica do Paraná, 2026. Material da disciplina.

RESCHKE, J. **The 'Basic' HTTP Authentication Scheme**. RFC 7617. [*S. l.*]: Internet Engineering Task Force, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7617. Acesso em: 6 nov. 2026.

TANENBAUM, A. S.; FEAMSTER, N.; WETHERALL, D. J. **Redes de computadores**. 6. ed. Porto Alegre: Bookman, 2021.

APÊNDICE A – CÓDIGO-FONTE DO FIRMWARE

Arquivo `fechadura_web.ino`. O nome e a senha da rede Wi-Fi foram substituídos por valores genéricos.

```
// Fechadura eletrônica com RFID + servidor HTTP (ESP32)
// Ligação: RC522 SDA=D5 SCK=D18 MOSI=D23 MISO=D19 RST=D27 | Relé S=D32

#include <WiFi.h>
#include <WebServer.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Preferences.h>
#include <time.h>

// ===== Wi-Fi: troque pelo nome e senha da rede (rede aberta: senha "") =====
const char* SSID  = "NOME_DA_REDE";
const char* SENHA = "SENHA_DA_REDE";

// ===== Login da pagina web (HTTP Basic Auth) =====
const char* WEB_USUARIO = "admin";
const char* WEB_SENHA   = "1234";

// ===== Pinos e configuração =====
#define SS_PIN        5
#define RST_PIN       27
#define RELAY_PIN     32
#define TEMPO_ABERTA  3000   // ms que a fechadura fica destravada
#define MAX_CARTOES   20
#define MAX_LOG       30

MFRC522 rfid(SS_PIN, RST_PIN);
WebServer server(80);       // servidor HTTP na porta 80 (TCP)
Preferences prefs;          // memória flash: a lista sobrevive ao desligar

String cartoes[MAX_CARTOES];   // UIDs autorizados
String nomes[MAX_CARTOES];     // nome do dono de cada cartão
int totalCartoes = 0;

String logAcessos[MAX_LOG];    // mais recente primeiro
int totalLog = 0;

String ultimoUID = "";         // último cartão lido (para cadastrar)
bool portaAberta = false;
unsigned long abriuEm = 0;

// ---------- Utilitários ----------
String horaAtual() {
  struct tm t;
  if (getLocalTime(&t, 10)) {
    char buf[20];
    strftime(buf, sizeof(buf), "%d/%m %H:%M:%S", &t);
    return String(buf);
  }
  return "+" + String(millis() / 1000) + "s";   // sem internet: tempo desde que ligou
}
```

```
void registrar(String evento) {
  for (int i = min(totalLog, MAX_LOG - 1); i > 0; i--) logAcessos[i] = logAcessos[i -
1];
  logAcessos[0] = horaAtual() + " - " + evento;
  if (totalLog < MAX_LOG) totalLog++;
  Serial.println(logAcessos[0]);
}

// Evita que um nome digitado injete HTML na página
String escapar(String s) {
  s.replace("&", "&amp;"); s.replace("<", "&lt;"); s.replace(">", "&gt;");
s.replace("\"", "&quot;");
  return s;
}

int buscarCartao(String uid) {
  for (int i = 0; i < totalCartoes; i++) if (cartoes[i] == uid) return i;
  return -1;
}

// ---------- Memória flash ----------
void salvarCartoes() {
  prefs.begin("fechadura", false);
  prefs.clear();
  prefs.putInt("total", totalCartoes);
  for (int i = 0; i < totalCartoes; i++) {
    prefs.putString(("u" + String(i)).c_str(), cartoes[i]);
    prefs.putString(("n" + String(i)).c_str(), nomes[i]);
  }
  prefs.end();
}

void carregarCartoes() {
  prefs.begin("fechadura", true);
  totalCartoes = prefs.getInt("total", 0);
  for (int i = 0; i < totalCartoes; i++) {
    cartoes[i] = prefs.getString(("u" + String(i)).c_str(), "");
    nomes[i]   = prefs.getString(("n" + String(i)).c_str(), "");
  }
  prefs.end();
}

// ---------- Fechadura ----------
void abrirPorta(String origem) {
  digitalWrite(RELAY_PIN, HIGH);
  portaAberta = true;
  abriuEm = millis();
  registrar("LIBERADO - " + origem);
}

void lerCartao() {
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;

  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
    if (i < rfid.uid.size - 1) uid += " ";
```

```
  }
  uid.toUpperCase();
  ultimoUID = uid;

  int i = buscarCartao(uid);
  if (i >= 0) abrirPorta("Cartao " + uid + " (" + nomes[i] + ")");
  else        registrar("NEGADO - cartao desconhecido " + uid);

  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}

// ---------- Páginas HTTP ----------
// Pede usuario e senha antes de qualquer rota (HTTP Basic Auth)
bool autorizado() {
  if (server.authenticate(WEB_USUARIO, WEB_SENHA)) return true;
  if (server.hasHeader("Authorization"))   // tentou entrar com senha errada
    registrar("LOGIN RECUSADO - " + server.client().remoteIP().toString());
  server.requestAuthentication(BASIC_AUTH, "Fechadura", "Acesso negado: login
necessario");
  return false;
}

void voltarInicio() {
  server.sendHeader("Location", "/");
  server.send(303);
}

void paginaInicial() {   // GET /
  if (!autorizado()) return;
  String h = "<!DOCTYPE html><html lang='pt-br'><head><meta charset='utf-8'>"
             "<meta name='viewport' content='width=device-width,initial-scale=1'>"
             "<title>Fechadura</title><style>"
             "body{font-family:sans-serif;max-width:640px;margin:20px auto;padding:0
12px}"
             "button{padding:8px 14px;cursor:pointer}input{padding:6px;margin:2px}"
             "table{border-collapse:collapse;width:100%}td,th{border:1px solid
#ccc;padding:6px;text-align:left}"
             ".abrir{font-size:18px;background:#1e2761;color:#fff;border:0;border-
radius:6px}"
             "</style></head><body><h1>Fechadura eletronica</h1>";

  h += "<p>Status: <b>" + String(portaAberta ? "DESTRAVADA" : "TRAVADA") + "</b></p>";
  h += "<form method='POST' action='/abrir'><button class='abrir'>Abrir
porta</button></form>";

  h += "<h2>Cartoes autorizados</h2><table><tr><th>UID</th><th>Nome</th><th></th></tr>";
  for (int i = 0; i < totalCartoes; i++) {
    h += "<tr><td>" + cartoes[i] + "</td><td>" + escapar(nomes[i]) + "</td><td>"
         "<form method='POST' action='/apagar' style='margin:0'>"
         "<input type='hidden' name='uid' value='" + cartoes[i] + "'>"
         "<button>Apagar</button></form></td></tr>";
  }
  if (totalCartoes == 0) h += "<tr><td colspan='3'>Nenhum cartao cadastrado</td></tr>";
  h += "</table>";

  h += "<h2>Cadastrar cartao</h2><p>Aproxime o cartao do leitor e clique em \"Usar
ultimo lido\".</p>"
```

```
       "<form method='POST' action='/cadastrar'>"
       "<input id='uid' name='uid' placeholder='UID' required>"
       "<button type='button'
onclick=\"fetch('/ultimo').then(r=>r.text()).then(t=>{uid.value=t||'';if(!
t)alert('Nenhum cartao lido ainda')})\">Usar ultimo lido</button><br>"
       "<input name='nome' placeholder='Nome do dono' maxlength='30' required>"
       "<button>Cadastrar</button></form>";

  h += "<h2>Log de acessos</h2><table id='log'>";
  for (int i = 0; i < totalLog; i++) h += "<tr><td>" + escapar(logAcessos[i]) +
"</td></tr>";
  if (totalLog == 0) h += "<tr><td>Sem registros</td></tr>";
  h += "</table><p><a href='/'>Atualizar</a></p>"

"<script>setInterval(()=>fetch('/log').then(r=>r.text()).then(t=>document.getElementById
('log').innerHTML=t),2000)</script>"
       "</body></html>";

  server.send(200, "text/html", h);
}

void rotaAbrir() {       // POST /abrir
  if (!autorizado()) return;
  abrirPorta("Navegador (" + server.client().remoteIP().toString() + ")");
  voltarInicio();
}

void rotaCadastrar() {   // POST /cadastrar
  if (!autorizado()) return;
  String uid = server.arg("uid");  uid.trim();  uid.toUpperCase();
  String nome = server.arg("nome"); nome.trim();
  if (uid.length() == 0 || nome.length() == 0) { server.send(400, "text/plain", "UID e
nome sao obrigatorios"); return; }
  if (buscarCartao(uid) >= 0) { server.send(409, "text/plain", "Cartao ja cadastrado");
return; }
  if (totalCartoes >= MAX_CARTOES) { server.send(507, "text/plain", "Limite de cartoes
atingido"); return; }

  cartoes[totalCartoes] = uid;
  nomes[totalCartoes] = nome;
  totalCartoes++;
  salvarCartoes();
  registrar("Cartao cadastrado: " + uid + " (" + nome + ")");
  voltarInicio();
}

void rotaApagar() {      // POST /apagar
  if (!autorizado()) return;
  String uid = server.arg("uid");
  int i = buscarCartao(uid);
  if (i < 0) { server.send(404, "text/plain", "Cartao nao encontrado"); return; }

  String nome = nomes[i];
  for (int j = i; j < totalCartoes - 1; j++) { cartoes[j] = cartoes[j + 1]; nomes[j] =
nomes[j + 1]; }
  totalCartoes--;
  salvarCartoes();
  registrar("Cartao removido: " + uid + " (" + nome + ")");
```

```
  voltarInicio();
}

void rotaLog() {         // GET /log  (a pagina pede a cada 2 s: polling)
  if (!autorizado()) return;
  String h = "";
  for (int i = 0; i < totalLog; i++) h += "<tr><td>" + escapar(logAcessos[i]) +
"</td></tr>";
  if (totalLog == 0) h += "<tr><td>Sem registros</td></tr>";
  server.send(200, "text/html", h);
}

void rotaUltimo() {      // GET /ultimo
  if (!autorizado()) return;
  server.send(200, "text/plain", ultimoUID);
}

// ---------- Programa principal ----------
void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);   // começa travada

  SPI.begin();
  rfid.PCD_Init();
  rfid.PCD_SetAntennaGain(rfid.RxGain_max);

  carregarCartoes();
  Serial.printf("%d cartoes carregados da memoria\n", totalCartoes);

  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);
  WiFi.begin(SSID, SENHA);
  Serial.print("Conectando ao Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nConectado! Acesse: http://" + WiFi.localIP().toString());

  configTime(-3 * 3600, 0, "pool.ntp.org");   // horário de Brasília (se tiver internet)

  server.on("/",          HTTP_GET,  paginaInicial);
  server.on("/abrir",     HTTP_POST, rotaAbrir);
  server.on("/cadastrar", HTTP_POST, rotaCadastrar);
  server.on("/apagar",    HTTP_POST, rotaApagar);
  server.on("/ultimo",    HTTP_GET,  rotaUltimo);
  server.on("/log",       HTTP_GET,  rotaLog);
  server.begin();

  registrar("Sistema iniciado");
}

void loop() {
  server.handleClient();
  lerCartao();

  // Trava de novo depois do tempo, sem travar o servidor com delay()
  if (portaAberta && millis() - abriuEm >= TEMPO_ABERTA) {
    digitalWrite(RELAY_PIN, LOW);
    portaAberta = false;
```

```
  }
}
```